

INF2010 Structures de données et algorithmes

Travail pratique 2 ***Mappes et dictionnaires***

Table des matières

1	Objectifs	2
2	Matériel fourni	2
3	Exercices	3
4	Remise	4

1 Objectifs

Ce laboratoire fait suite au cours 03 sur le hachage et les tables de dispersion (autrement « tables de hachage » ou « hash tables »), en suivant les exemples du chapitre 5 de Weiss (*Data Structures and Algorithm Analysis in Java*, 2e ed., 2006). À la fin de ce laboratoire, vous devriez être capable de :

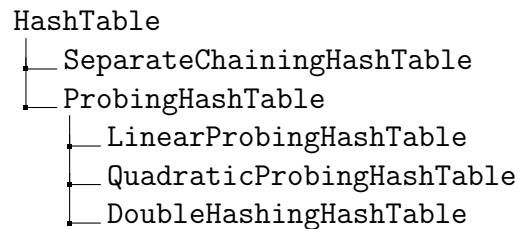
- implémenter et instrumenter différentes stratégies de résolution de collisions (chaînage, sondage quadratique) ;
- concevoir une fonction de dispersion adaptée à un type de clé donné et en évaluer la qualité ;
- mesurer empiriquement, à l'aide de graphiques, l'effet du facteur de charge sur la performance de chaque stratégie ;
- comparer et interpréter de façon critique la performance relative du chaînage et du sondage quadratique.

2 Matériel fourni

Les fichiers suivants vous sont fournis comme point de départ. Ils contiennent les implémentations vues en cours (chaînage et sondage quadratique), basées sur le manuel de référence du cours de Weiss (chapitre 5).

- `HashTable.java` — la classe abstraite pour une table de dispersion.
- `SeparateChainingHashTable.java` — table de dispersion avec résolution des collisions par chaînage (listes chaînées).
- `ProbingHashTable.java` — une autre classe abstraite qui définit des méthodes utilisées par les trois types de table de dispersion avec résolution des collisions par sondage.
- `LinearProbingHashTable.java` — table de dispersion avec résolution des collisions par sondage linéaire.
- `QuadraticProbingHashTable.java` — table de dispersion avec résolution des collisions par sondage quadratique.
- `DoubleHashingHashTable.java` — table de dispersion avec résolution des collisions par double adressage.
- `studentXX.zip` — un archive ZIP contenant des classes compilées que vous allez tester en « boîte noire ». Il y a plusieurs dossiers. Choisissez celui correspondant à votre numéro d'équipe de laboratoire sur Moodle. Par exemple, si vous étiez membre du groupe « Gr2_03 », vous choisirez le dossier `student03.zip` ; si vous étiez membre de « Gr3_32 », vous choisirez le dossier `student32.zip` et ainsi de suite.

Pour résumer, les fichiers `java` contiennent des classes qui sont organisées hiérarchiquement de la manière suivante :



Ainsi, dans la suite du document, « les quatre types de table » indique les quatre classes non abstraites, c'est-à-dire `SeparateChainingHashTable`, `LinearProbingHashTable`, `QuadraticProbingHashTable`, et `DoubleHashingHashTable`.

3 Exercices

Exercice 1 — Instrumentation de la table

Implémentez `findPos(x)` (la fonction de sondage) dans chacun de `LinearProbingHashTable` et `DoubleHashingHashTable` comme indiqué. Notez que vous devez aussi entrer votre matricule étudiant dans `DoubleHashingHashTable`, ce qui va vous donner une fonction personnalisée !

Dans la classe `HashTable`, vous verrez les méthodes abstraites pour déterminer le facteur de charge, le nombre de rehash, la taille totale de la table (en termes de la taille de la structure principale), et le nombre de collisions observées. Implémentez-les, soit dans cette classe (en enlevant « `abstract` ») soit dans ses sous-classes, selon les besoins de votre implémentation.

Exercice 2 — Comparaison expérimentale des stratégies

Dans cet exercice, vous allez comparer différentes tables selon plusieurs métriques : la taille totale de la table, le nombre de rehash, le temps total, le facteur de charge, et le nombre de collisions.

Si vous avez bien implémenté les méthodes d'Exercice 1, en lançant chacun des quatre programmes, vous entrerez un nombre d'éléments à ajouter dans votre table, et le programme affichera les valeurs suivantes. Pour chacun des quatre types de table, construisez une table avec les résultats obtenus, de la forme indiquée dans Table 1. Notez que le nombre d'éléments ajoutés augmente selon des multiples croissants de puissances de 10.

Ajoutez assez de lignes avec une assez grande gamme de nombres d'éléments insérés pour démontrer des tendances dans chaque cas. Commentez ces tendances, en ajoutant cinq graphiques clairs et explicatifs (avec la première colonne ci-dessus comme axe horizontal), un pour chaque métrique, qui comparent les quatre tables différentes. Est-ce que les tendances sont en rapport avec la théorie ? Est-ce que c'est toujours le cas ? Pourquoi ou pourquoi pas ?

Éléments ajoutés	Taille totale	Rehash	Temps (ns)	Facteur de charge	Collisions
100					
200					
300					
⋮					
1000					
2000					
3000					
⋮					
100 000 000					

TABLE 1 – Forme de la table à soumettre

Exercice 3 — Comparaison des stratégies à boîte noire

Dans le dossier donné, vous trouverez trois fichiers qui implémentent d'autres variations sur les tables de dispersion. Vous n'avez que des fichiers jar (pas java), et vous n'avez pas nécessairement tous les mêmes fichiers que les autres étudiants. Lancez chacun dans le terminal avec la commande `java -jar AlgorithmeX.jar`, et vous verrez les mêmes données imprimées que dans Exercice 2. Créez une table et des graphiques ici aussi, de la même manière. Commentez. Quel algorithme choisiriez-vous ? Est-ce que votre réponse dépend des autres paramètres (p. ex., nombre d'insertions) ?

4 Remise

Déposez les éléments suivants sur Moodle avant la date limite indiquée :

- Le code source complet et commenté des fichiers modifiés de l'Exercice 1 :
 - `SeparateChainingHashTable.java`
 - `ProbingHashTable.java`
 - `LinearProbingHashTable.java`
 - `QuadraticProbingHashTable.java`
 - `DoubleHashingHashTable.java`
 - `HashTable.java`
- Un court rapport (2 à 3 pages) contenant les tables et graphiques demandés aux Exercices 2 et 3 avec vos commentaires.

Note

L'IA générative a été utilisée dans la conception de ce fichier, principalement pour extraire le code des diapositives du cours et pour générer les algorithmes dans Exercice 3.